



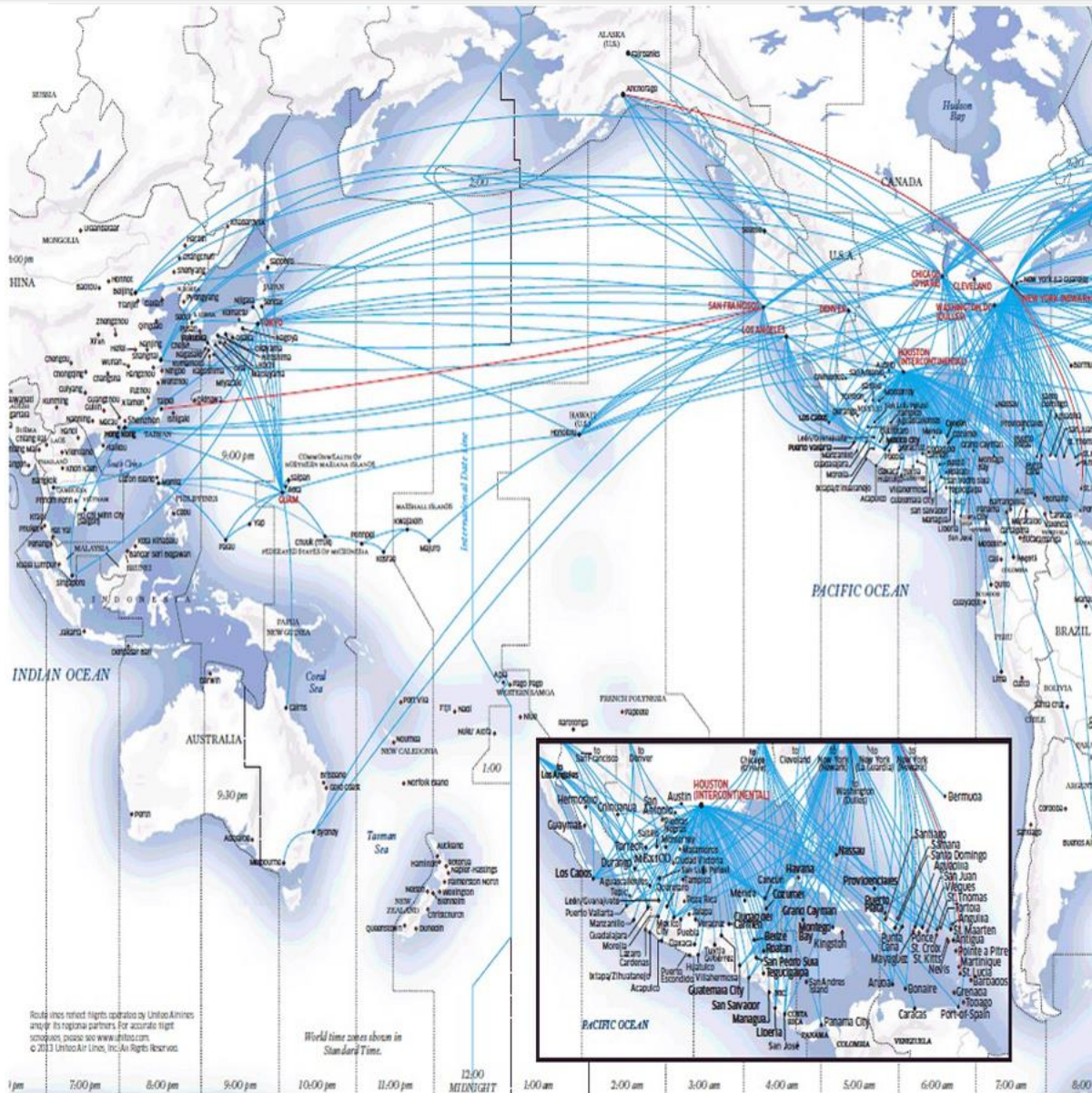
Module4—Graph

1. กราฟคืออะไร ทำไมต้องใช้โครงสร้างข้อมูลแบบกราฟ
2. นิยามของกราฟ digraph, undigraph, vertice, edge, adjacent, incident, degree, path
3. การเก็บกราฟด้วย adjacency list
4. การท่องกราฟด้วย BFS



ข้อมูล

- มีทิศทาง
- มีระยะทาง





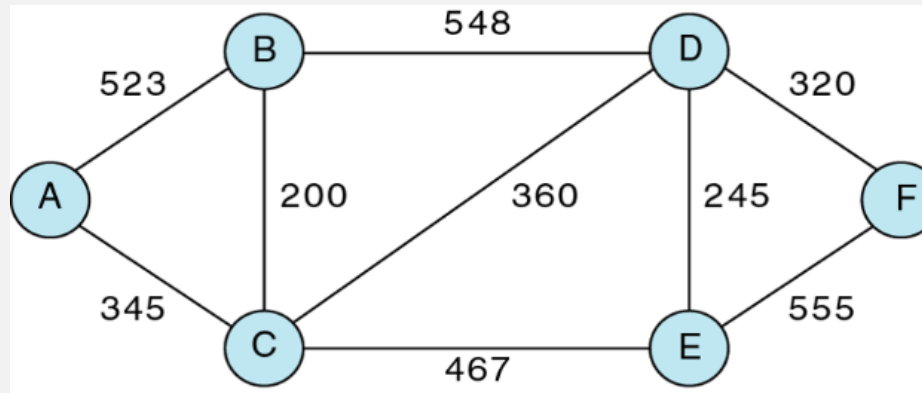
โครงสร้างข้อมูลแบบกราฟ

นิยมใช้กับงานที่ข้อมูลที่แน่น ความสัมพันธ์ระหว่างสิ่งต่าง ๆ

- **Google Maps / GPS** หาเส้นทางที่สั้นที่สุด
- **Social Network** แนะนำเพื่อน, วิเคราะห์เครือข่าย วิเคราะห์ความสัมพันธ์ของสังคม เช่นใครเป็นคนสำคัญในเครือข่าย
- **การวาง Network** (ข่ายงานคอมพิวเตอร์)
- **การวิเคราะห์เส้นทางวิกฤติ** จุดคอขวด



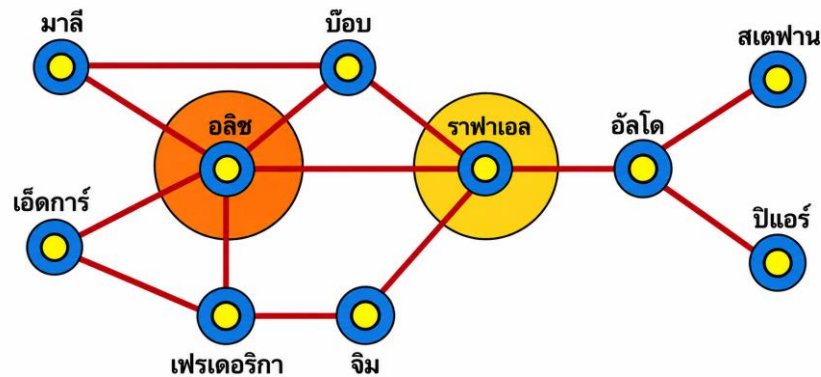
การหาระยะทางที่สั้นที่สุด



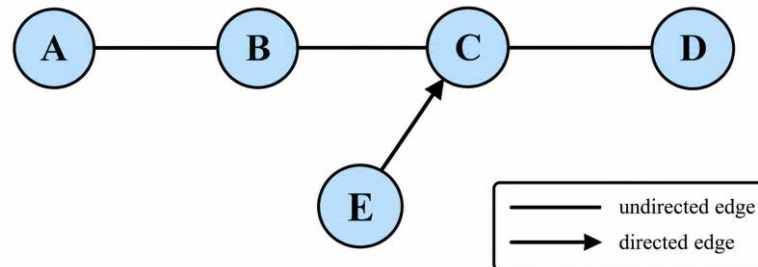


Social Network

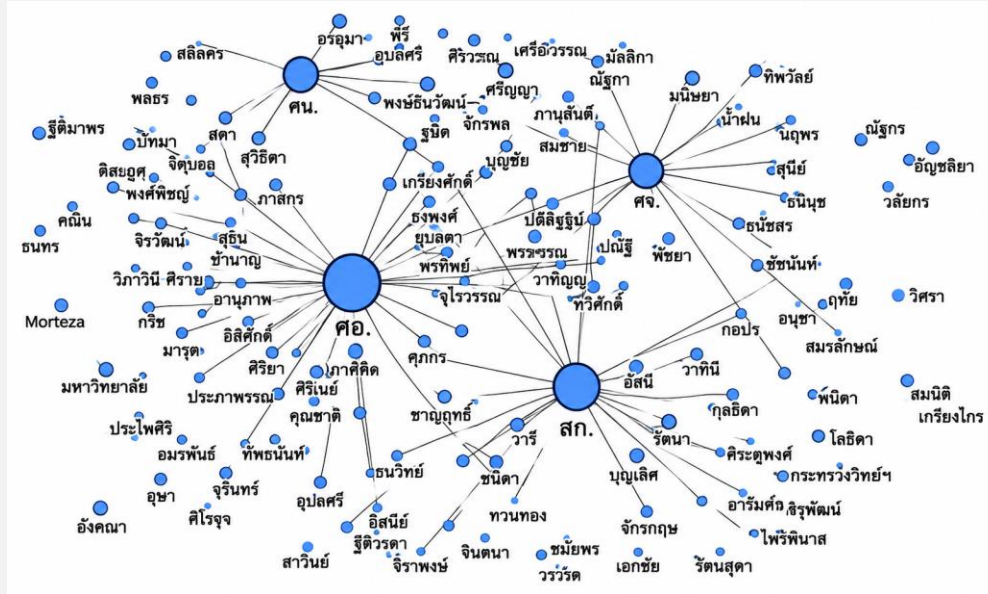
1. ใครเป็นคนสำคัญในเครือข่าย — Centrality



2. หา "คนกลาง" ที่เชื่อมคนหลายกลุ่ม



3. หา Community — ใครอยู่กลุ่มเดียวกัน

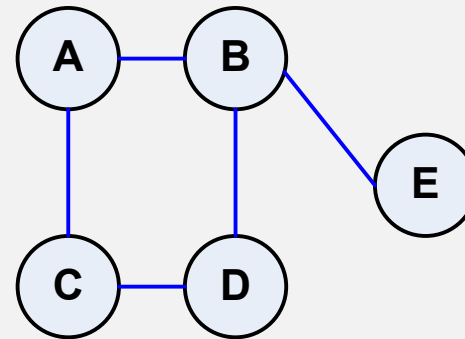
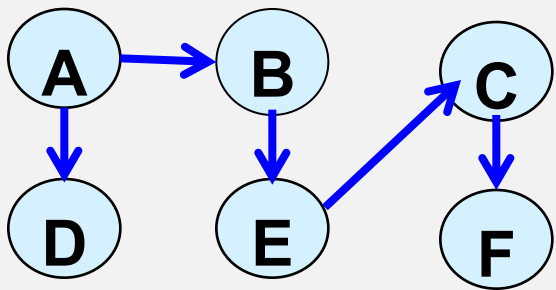


4. Friend Recommendation — แนะนำเพื่อน, โฆษณา



4.1 Definition

1. directed graph(digraph)
2. Undirected graph(Undigraph)



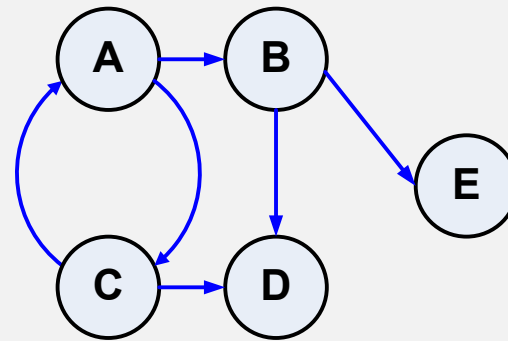


4.1 Definition

1) A directed graph(or digraph) G : is a pair (V, E) , where V is a finite set and E is binary relation on V .

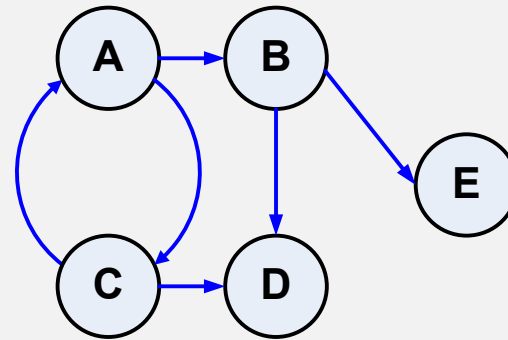
- The **set V** is called the **vertex set** of G , and its elements are called vertices.

$$V = \{A, B, C, D, E\}$$





- The **set E** is called the **edge set** of G , and its elements are called edges.



The edge set E consists of ordered pairs of vertices

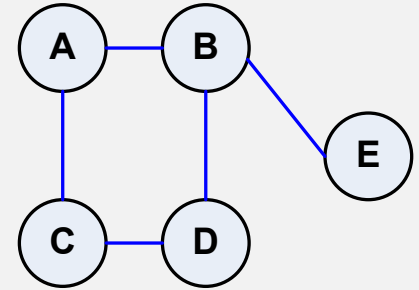
$$V = \{A, B, C, D, E\}$$

$$E = \{ (A, B), (A, C), (B, D), (B, E), (C, A), (C, D) \}$$



2) An undirected graph G :

$G = (V, E)$, the edge set E consists of unordered pairs of vertices, rather than ordered pairs.



$$V = \{A, B, C, D, E\}$$

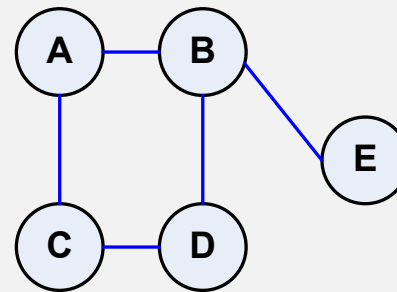
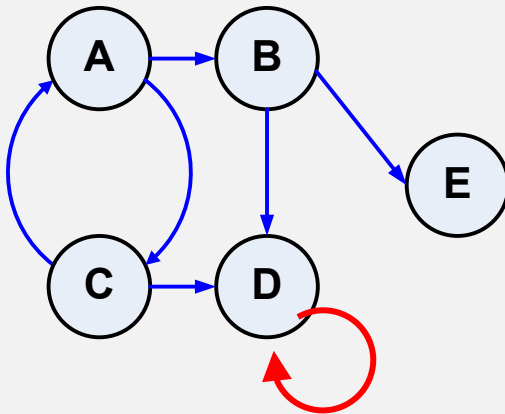
$$E = \{ (A, B), (B, A), (A, C), (C, A), (B, D), (D, B), (B, E), (E, B), (C, D), (D, C) \}$$



3) self-loops : edges from a vertex to itself.

Undigraph self loop are forbidden, and so every edge consists of exactly two distinct vertices.

Cycle : a cycle in a graph is a non-empty trail in which only the first and last vertices are equal



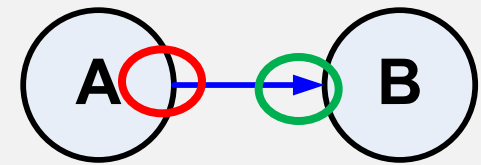


4) Incident :

digraph : if (u,v) is an edge in a directed graph $G=(V,E)$, we say that (u,v) is

incident from or leaves vertex u

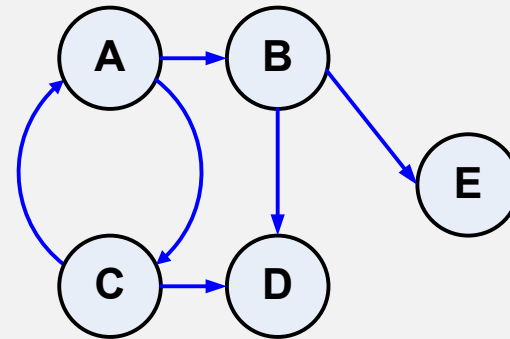
incident to or enters vertex v .



Question edge (B,E)

Incident from

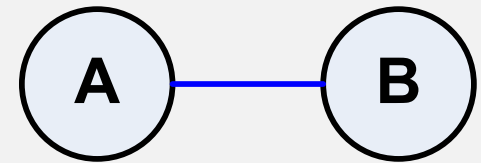
Incident to





undigraph : if (u,v) is an edge in a directed graph $G=(V,E)$, we say that (u,v) is **incident on** vertices u and v

is **incident on** vertices u and v



Question edge (A,B)

Incident from

Incident to

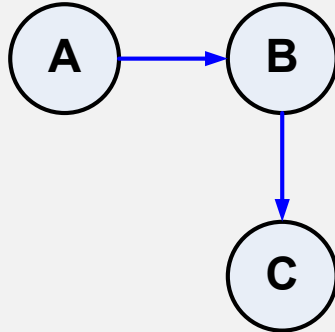
Edge (A,B) is incident on vertices A and B



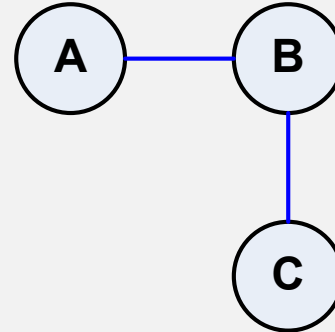
5) adjacent

Digraph : If (u,v) is an edge in a graph $G=(V,E)$, we say that v is adjacent to vertex u . If v is adjacent to u denote by $u \rightarrow v$

Undigraph : Adjacent relation is symmetric.



B ประชิด A $A \rightarrow B$



B ประชิด A และ A ประชิด B



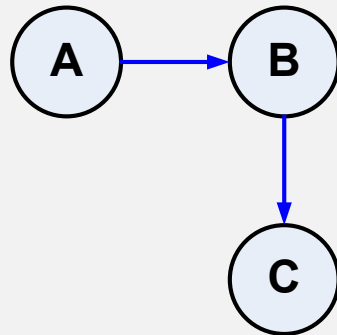
6) Degree

Digraph : in degree

(number of edges entering it.),

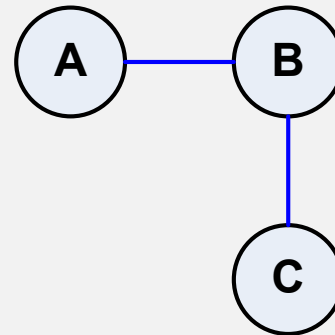
out degree

(Opposite in degree.)



A in degree = 0
A out degree = 1

Undigraph : is the number of edges incident on it.



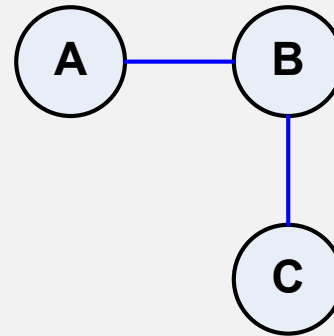
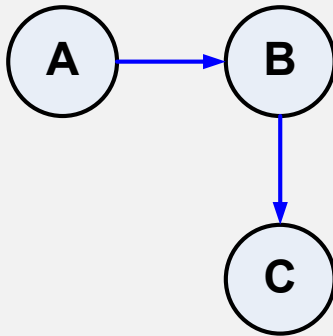
A degree = 1

B degree = 2

C degree = 1



7) path of length : Path k from a vertex u to a vertex u' in a graph $G=(V,E)$ is a sequence $(v_0, v_1, \dots, v_k)$ of vertices such that $u = v_0$, $u' = v_k$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \dots, k$





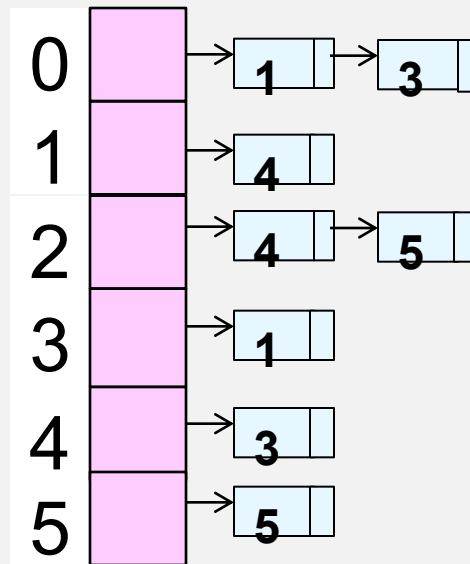
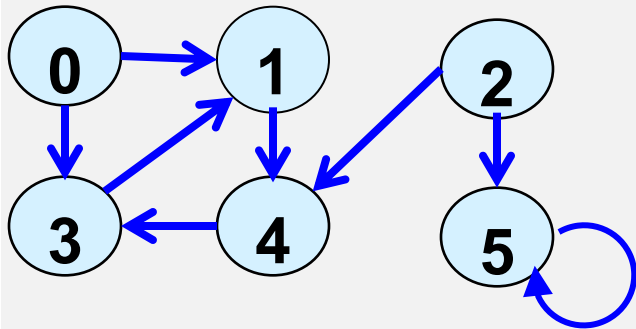
4.2 โครงสร้างข้อมูลที่ใช้เก็บข้อมูลในรูปแบบกราฟคือ

1. Adjacency list
2. Adjacency Matrix

โครงสร้างข้อมูลทั้ง 2 แบบนี้จะสามารถเก็บกราฟได้ทั้ง digraph และ undigraph



Digraph

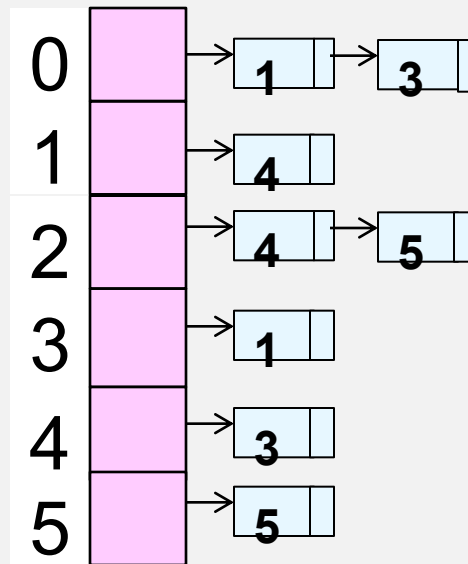
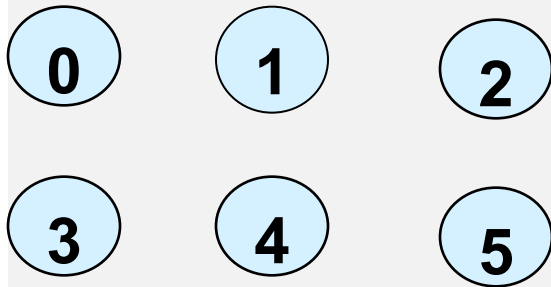


	0	1	2	3	4	5
0	0	1	0	1	0	0
1	0	0	0	0	1	0
2	0	0	0	0	1	1
3	0	1	0	0	0	0
4	0	0	0	1	0	0
5	0	0	0	0	0	1

1. Adjacency list
2. Adjacency matrix



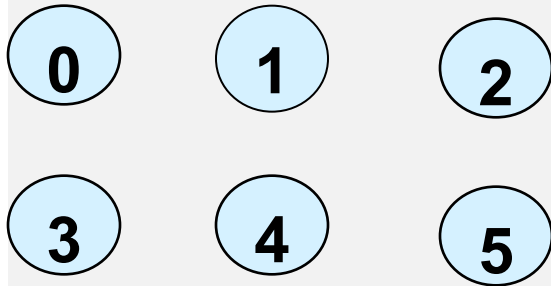
Digraph



1. Adjacency list
2. Adjacency matix



Digraph

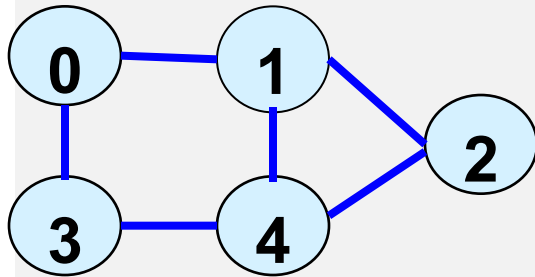


	0	1	2	3	4	5
0	0	1	0	1	0	0
1	0	0	0	0	1	0
2	0	0	0	0	1	1
3	0	1	0	0	0	0
4	0	0	0	1	0	0
5	0	0	0	0	0	1

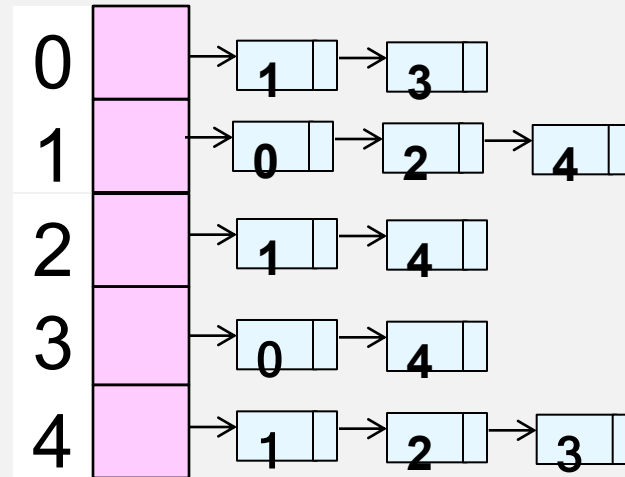
1. Adjacency list
2. Adjacency matix

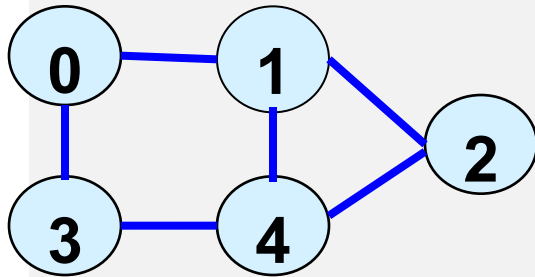


Undigraph



Adjacency list





Adjacency Matrix

	0	1	2	3	4
0	0	1	0	1	0
1	1	0	1	0	1
2	0	1	0	0	1
3	1	0	0	0	1
4	0	1	1	1	0



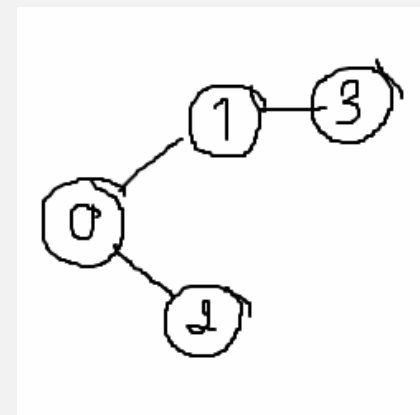
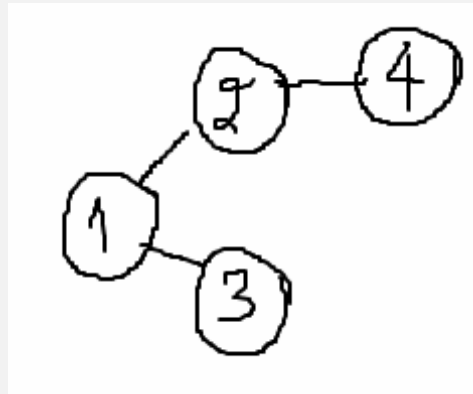
การ Implement adjacency list ด้วย Vector

Input

4	3
1	2
2	4
1	3

Output

0 : 1	2
1 : 0	3
2 : 0	
3 : 1	





```
#include <iostream>
#include <vector>
using namespace std;

#define MAX_N 100000

int n, m; //vertices edges
vector<int> adj[MAX_N];
int deg[MAX_N];

void init()
{
    for(int i=0;i<n;i++)
        deg[i]=0;
}
```

```
void read_input()
{
    int u,v;
    cout << "#node #edge : \n";
    cin >> n >> m;
    for(int i=0;i<m;i++)
    {
        cin >> u >> v; u--;v--;
        adj[u].push_back(v);
        deg[u]++;
        adj[v].push_back(u);
        deg[v]++;
    }
}
```



```
void show()
{
    for(int i=0;i<n;i++)
    {
        cout << i << ":";
        for(int j=0;j<deg[i];j++)
            cout << adj[i][j] << " ";
        cout << endl;
    }
}

int main()
{
    init();
    read_input();
    show();
}
```



การบ้าน ข้อ 1

Input Undigraph

4 3

1 2

2 4

1 3

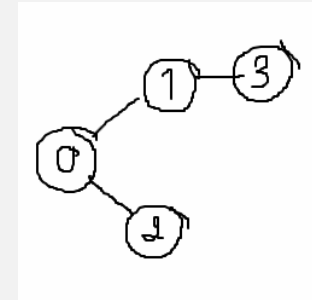
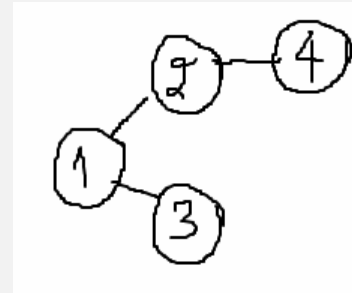
Output

1 : 2 3

2 : 1 4

3 : 1

4 : 2





การบ้าน ข้อ 2

Input Digraph

6 8

1 2

1 4

2 5

.....

6 6

Output

1 : 2 4

2 : 5

...

6 : 6

